

Carleton University
Department of Systems and Computer Engineering
SYSC 3303A - RealTime Concurrent Systems - Winter 2026

Project Specification: Firefighting Drone Swarm

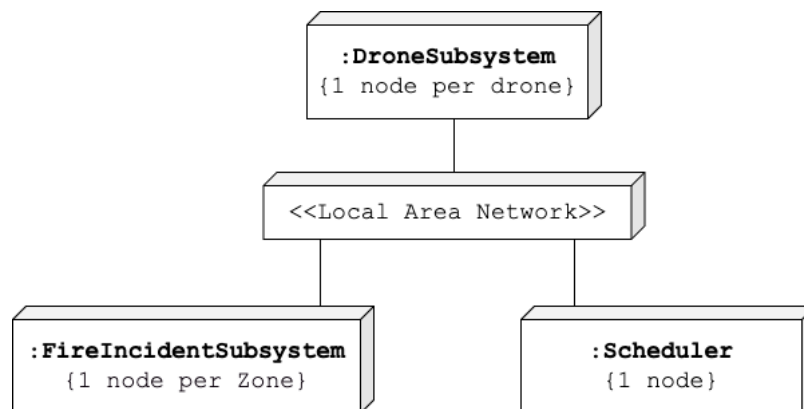
Teams will design and implement a control system and simulator for a firefighting drone swarm. The system will include:

1. A **Scheduler** (controlling and coordinating the drones)
2. A **Drone Subsystem** (representing the drones themselves, including motors/propellers, water/foam tanks, and sensors)
3. A **Fire Incident Subsystem** (simulating one or more “zones” where fires can break out, along with sensors and user requests for assistance)

Each of these three components should run as a separate process (i.e., separate Java programs) and communicate using **DatagramSocket** objects (UDP). Eventually, they will be deployed on separate computers in the lab, or on a single machine with multiple Java instances.

Your simulation must be **configurable** with respect to the number of drones, the number of fire zones, the time it takes for drones to drop firefighting agents, how long it takes to travel between zones, and so on. Your system is expected to operate in “**real-time**,” meaning that it should simulate the passage of time for drone movements, water/foam drops, nozzle operations, and so on.

The code will be written in **Java**, using the **IntelliJ IDE**, and must be designed to run within the provided lab environment.



System Components

1. Scheduler

- **Core Controller:** Receives fire incident requests from the Fire Incident Subsystem, determines which drone(s) to dispatch.
- **Fault & Error Handling:** Must detect problems such as a drone getting “stuck” in flight or a nozzle failing to open.
- **Dispatch & Coordination:** Sends commands to drones (take off, drop agent, return to base, etc.) and monitors their status.
- **Runs as its own process.**

2. Drone Subsystem

- **Water/Foam Tanks:** The drone carries a limited firefighting agent supply, which can be “dropped” on a fire.
- **Motors/Propellers:** Control flight between incident zones
- **Bay Doors / Nozzle:** Opens or activates to drop water/foam
- **Sensors & Indicators:**
 - **Arrival Sensor:** Detects when the drone has arrived at or is above a target fire zone.
 - **Agent Level Sensor:** Indicates remaining water/foam capacity.
 - **Status Lights:** e.g., “flying,” “dropping agent,” “refilling,” etc.

Each drone subsystem instance communicates with the Scheduler to report status and receive commands.

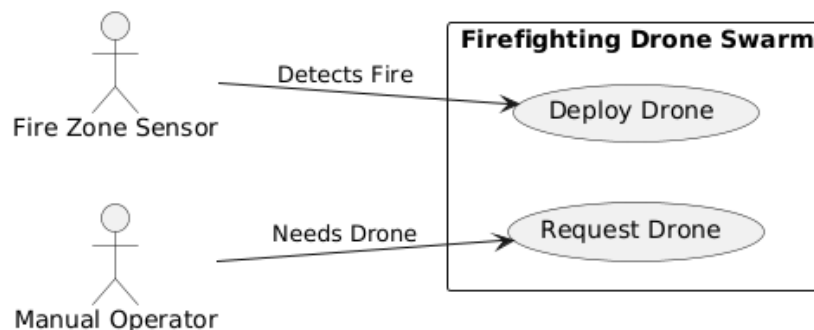
3. Fire Incident Subsystem

Simulates zones where fires occur.

- **Fire Request Mechanism:** When a fire starts (or flares up), it sends a request for drone assistance.
- **Incident Lamps/Indicators:** Indicate that a drone is inbound or currently fighting the fire.
- **Simulated “Users”:** Could be sensors or operators who report the fire. They also confirm when the fire is extinguished.

Generates arrival notifications, logs, or other relevant data to verify correct drone service.

Use Cases



Deploy Drone Use Case

Actors: Fire Zone Sensor (primary), Arrival Sensor

Precondition: A fire is detected at a given zone.

Description:

1. The fire zone sensor reports a fire to the Scheduler, providing its location and severity.
2. The fire incident sensor detects a fire and sends a “fire request” to the Scheduler.
3. The Scheduler chooses an available drone (or drones) and instructs it/them to proceed to the affected zone.
4. If the drone is on standby (idle), the system transitions the drone to “flight” mode (instructs it to depart immediately). Otherwise, the request is queued and awaits scheduling.
5. The Scheduler adds this request to its queue of missions.
6. The drone receives flight instructions, takes off, and travels toward the fire zone.
7. As the drone approaches (position is tracked in real-time), the arrival sensor signals the Scheduler.
8. The Scheduler commands the drone to initiate firefighting (drop water/foam).
9. The drone reduces its water/foam capacity during the drop. If more water/foam is needed, it may need to leave and refill.
10. Once the fire is controlled (is extinguished), the zone sensor notifies the Scheduler that the fire is out.
11. The drone either returns to base or proceeds to the next assignment.

Alternatives:

- If the drone cannot deploy its water/foam nozzle (fault), the Scheduler must **handle this error** and possibly dispatch another drone. The drone must be **returned back to base**.
- If the drone is out of water/foam, it must first **refill** (return back to base).

Postcondition: The zone is either fully serviced or awaiting further assistance from another drone.

Request Drone Use Case

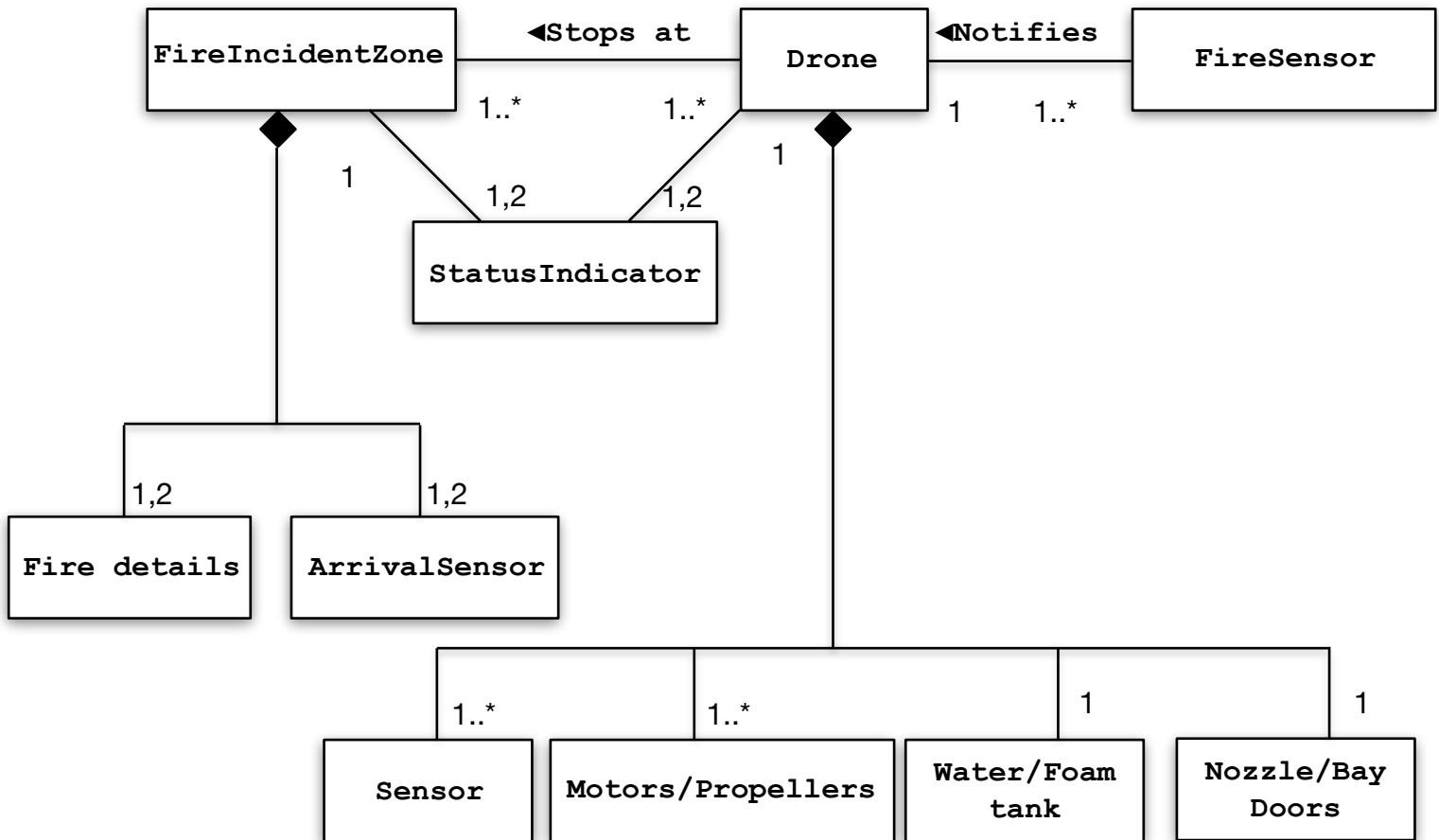
Actors: Manual Operator (primary), Arrival Sensor

Precondition: A user at a fire zone calls for drone assistance (manually or due to heightened risk).

Description:

1. A user or system operator at the zone triggers a “request drone” event.
2. The Fire Incident Subsystem sends the request to the Scheduler with details about the zone’s needs.
3. The Scheduler receives this request and identifies which drone is best suited (e.g., based on location, available water/foam, battery).
4. Upon receiving instructions, the drone navigates to the fire zone.
5. When the drone arrives (detected by an arrival sensor), the Scheduler is notified.
6. The drone drops water/foam. ~~or conducts a reconnaissance pass if instructed.~~
7. If multiple zones need attention, the drone can be routed accordingly.

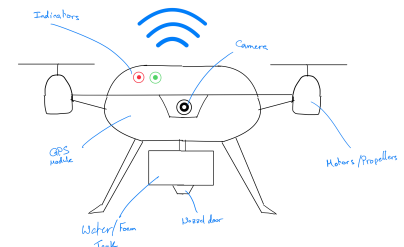
Postcondition: The drone has been successfully deployed to the requesting zone for immediate or pending firefighting action.



Static Model of the Problem Domain

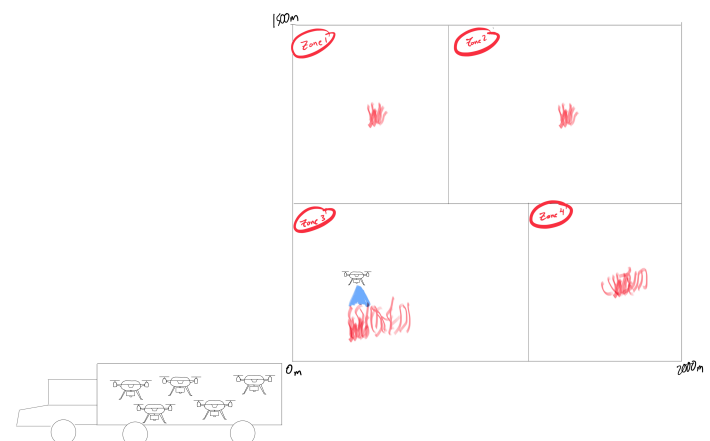
Each drone has:

- **Water/Foam tank:** limited capacity of firefighting agent.
- **Motors/Propellers:** allow movement between zones.
- **Nozzle/Bay Doors:** open/close mechanism for firefighting agent release.
- **Status indicators:** flight, approach, dropping agent, etc.
- **Sensors:** track arrival, measure water/foam levels, detect faults.



Each fire incident zone has:

- **Fire sensors or manual triggers** (Request buttons): to request a drone.
- **Indicators:** show that a drone is inbound, has arrived or actively dropping agent.
- **Arrival sensors:** detect drone presence.
- **Fire details:** severity, spread rate, etc.



Real-Time Behavior & Configurability

- You will simulate the time it takes for drones to fly between zones, drop water/foam, and possibly land or hover for extended operations.
- The system must accept configuration parameters such as:
 - Number of drones**
 - Number of zones (fire sites)**
 - Travel time** between zones
 - Agent drop time** (time required to release water/foam)
 - Drone capacity** (amount of agent each drone can carry)

Fire Incident Subsystem Simulation

The fire incident subsystem simulates fire outbreaks and potential user/automated requests. It should also test for the proper operation of the drone, i.e., a fire is detected and a request was sent to the drone, waits for the arrival of a drone to the zone, then extinguish the fire (release firefighting agent).

Your fire incident subsystem should read an input file that drives events, e.g., time stamps, zone IDs, severity levels, or repeated calls for assistance. The file should use the format shown below:

Time	Zone ID	Event type	Severity
hh:mm:ss.mmm	n	FIRE_DETECTED/DRONE_REQUEST	High/Moderate/Low
14:03:15	3	FIRE_DETECTED	High
14:10:00	7	DRONE_REQUEST	Moderate

i.e., a time stamp, white space, an integer representing the zone that the fire is detected or drone was requested in, white space, a string consisting of either FIRE_DETECTED or DRONE_REQUEST, a white space, a string consisting of either High, Moderate or Low. Use the following amounts (in litres) of water/foam needed for each event severity:

Severity	Amount of water/foam (L)
Low	10
Moderate	20
High	30

For example, in the table above, a fire is detected at 2:03:15 pm in zone 3 with high severity. It is strongly advisable to maintain separate queues for each zone and severity since fires might be at different locations before a certain drone reaches its destination. The subsystem must handle multiple simultaneous fire requests (in case fires erupt in different zones). It must confirm correct drone action: a fire zone receives assistance after a drone arrives and uses its agent.

Also assume that all zones are rectangular shapes and their coordinates (in meters) must be read from a different file that follows the format shown below:

Zone ID	Zone Start	Zone End
n	(x1, y1)	(x2, y2)
1	(0, 0)	(700,600)
2	(0, 600)	(650,1500)

Scheduler Specification

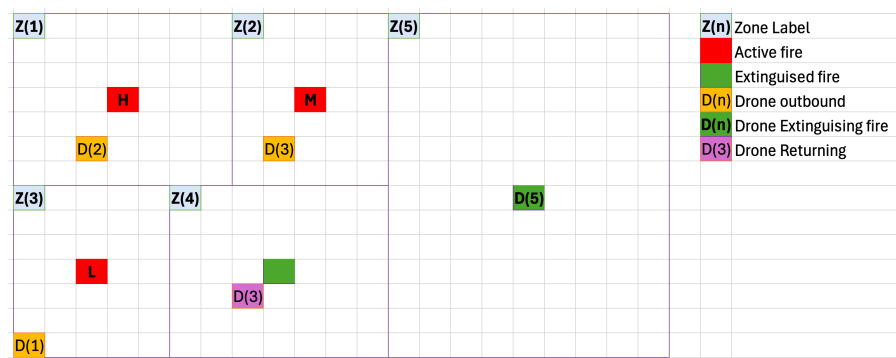
- Accepts fire reports (or requests) from the Fire Incident Subsystem and dispatches drones to respond.
- Manages mission queues to minimize response times or prevent any zone from being ignored.
- Tracks and updates each drone's status: location, remaining firefighting agent, faults, etc.
- Must handle communication faults (lost packets) or drone hardware faults (nozzle failure, drone stuck in flight).

Drone Subsystem Specification

- Implements the drone state machine: idle, en route, dropping agent, refilling, faulted, etc.
- Sends notifications to the Scheduler upon arrival at a zone.
- Opens/closes nozzle or bay doors when instructed by the Scheduler (or upon local state changes).
- Tracks agent capacity and simulates depletion during each firefighting operation. If empty, the drone must refill.

User Interface

- **Minimal** Graphical User Interface (GUI) is required for early iterations; eventually, show a **map/console** of zones and drones. Refer to the example below
- Must allow loading of the **incident input file** and system configuration:
 - Number of drones, number of zones, agent capacity, travel times, etc.
- The final iteration should display each drone's position/status in real time.



Development Process

An **iterative, incremental process** will be used to develop the system. Each iteration produces an executable subset of the final solution. During lab sessions, you will present the working features of your **previous** iteration's code.

Iteration 0 – Data Collection

In this iteration you will collect some data regarding different types of available drones and some of their specifications. A drone must be able to carry at least 10 kg and a maximum 15kg load.

Collect or estimate real-world drone parameters:

- Travel time between zones
- Time to open/close nozzle doors
- Rate of dropping water/foam
- Acceleration/deceleration for takeoff/landing

Record this data (e.g., in Excel), and justify how you arrived at your chosen values. Add references to these values. Submit a PDF with analysis.

Iteration 1 – Establish Basic Communication

Using Assignment 1, create three threads, one for each subsystem identified above. The Fire Incident Subsystem and the Drones are the clients in the system; the Scheduler is the server. The Fire Incident subsystem is to read in events using the format shown above: Time, Zone ID, Event Type and Severity. Each line of input is to be sent to the Scheduler. The Drones will make calls to the Scheduler which will then reply when there is work to be done. The Drone will then send the data back to the Scheduler who will then send it back to the Fire Incident subsystem. For this iteration the Scheduler is only being used as a communication channel from the Fire Incident Subsystem thread to the Drone thread and back again. It is only necessary to create a test case showing that your program can read the input file and pass the data back and forth. Note that in a future iteration, there will be multiple Drones all running independently of each other, and there will be three programs running on separate computers, so be sure that you allow for this configuration when you design your code, or you will have to refactor your design at a later point in time.

You probably don't want to pass text strings from the Fire Incident subsystem to the rest of the system. You should develop a data structure which only passes the necessary information representing the hardware device (Fire Zone ID, Severity, etc.) to the scheduler. If you choose to work ahead, it would be a good idea to tag this "version" in GIT for the purposes of the demo to the TA's and for submission in Brightspace.

Iteration #1 Summary:

- Create three threads (or processes):
 1. **Fire Incident Subsystem** (client)
 2. **Drone Subsystem** (client)
 3. **Scheduler** (server)
- The Fire Incident Subsystem reads input events (time, zone, event type, severity, etc.) and sends them to the Scheduler.
- The Drone Subsystem consults the Scheduler to see if it has tasks to perform (put out fires).
- For this iteration, the Scheduler primarily acts as a pass-through: reading messages from the Fire Incident Subsystem and forwarding them to the Drone Subsystem (and vice versa).
- Focus on communication: verify you can read an input file, pass messages, and get round-trip acknowledgments. Real scheduling logic will be added later.
- Create the structure for the Graphical User Interface (GUI) similar to the example below using swing library.

Previous questions on Iteration #1:

Below are some of the questions students asked in Winter 2025 regarding iteration 1:

- Are we considering more than 1 incident?

- You can start with one incident for this iteration, however, it is recommended to start with your input file processing method/function that can read until the end of the file.
- Should the system prioritize sending drones to higher severity incidents or should the drones go to incidents as they come in indicated by the input file?
 - Incidents should be processed as they arrive to the system (arrival time in the input file). The severity is the amount of agent (water/foam) that is needed.
- Drone must be able to carry 10kg/10L and up to 15kg/15L. Should it always carry 15kg so it can help extinguish other fires? Or should the drone carry as much as needed to extinguish the scheduled event? I.e. LOW severity only carry 10L?
 - A drone should carry the maximum amount of payload it can carry. Remember to include the weight of the water reservoir, nozzle, and any other components in the calculation of the payload. How many drones to send should be part of your scheduling algorithm. I recommend that a drone can go to multiple zones if they still have some agent left. Example: if a drone is carrying 14L of agent and they are sent to a low severity zone (10L) and they have enough battery so they can return back to the base, then they should be sent to any nearby zones to use the remaining 4L.
- Are we expected to use Datagram communication (UDP) for data transfer between these components at this stage of the project?
 - For this iteration, communication should be in a similar manner as Assignment 01. There is no UDP communication yet. UDP will be implemented in Iteration 03.
- Does the Drone Subsystem need to send a message back to the Fire Incident Subsystem?
 - The fire incident subsystem sends events to the scheduler and the scheduler sends them to drones. Once a drone services a certain zone, they send a confirmation of finishing to the scheduler which notifies the fire incident subsystem.
- What exactly is the Drone Subsystem consulting the Scheduler for? Does "consulting" mean that the drone sends data back to the Scheduler for acknowledgment, or is it simply requesting task assignments?
 - A drone will be idle and when there is a fire in the queue of the scheduler, it should send it to the drone. Drone must confirm when they finish that task.
- Are we expected to simulate the fire-extinguishing process in this iteration? Specifically, should we implement functionality for sending the drone to an incident zone and putting out the fire? Or is that a requirement for a later iteration?
 - Yes, you expected to implement the function of sending the drone extinguishing and returning. This must be done as a calculation. So you need to derive a formula based on values collected in iteration 0.
- Should your zones/map include buildings/obstacles/terrain?
 - No, there shouldn't be any buildings in the map. You can have that in future work section of the final report. However, your code should take into account possible collisions between drones.
- Could we also use UML diagrams generated by IntelliJ IDE?
 - UML diagrams should be one of the first steps of your project(static modelling) before starting your coding. Also, the purpose of the project is to practice all the topics that we have covered in this course and apply what you have learned in previous courses. You are expected to create the UML diagrams for your project.
- Is there a file type that's preferred for input file? csv? txt?
 - Input file type should be .csv (comma separated values)

Work Products for Iteration #1:

- “README.txt” file explaining the names of your files, set up instructions, etc.
- Breakdown of responsibilities of each team member for this iteration
- UML class diagram and sequence diagrams.
- Detailed set up and test instructions, including test files used
- Code (.java files, all required IntelliJ files, etc.)

Iteration 2 – Add Scheduling & Drone State Machines

The goal of this iteration is to add the state machines for the scheduler and drone subsystems assuming that there is only one drone. However, you should bear in mind that for the next iteration, your system is expected to coordinate between the drones in order to maximize the number of zones serviced over time (i.e., the throughput). Note that the drone subsystem is used to notify the scheduler that an drone has reached a zone, so that once an drone has been told to move, the drone subsystem also has to be informed so that it can send out messages back to the scheduler to denote the arrival by an drone. You can either maintain a single event list or have separate tasks for each drone. Perhaps you can think of another way of doing it too.

Iteration #2 Summary:

- Implement core scheduling logic to decide which drone handles a new fire.
- Implement drone state transitions (idle, en route, dropping agent, etc.).
- Start with only one drone to simplify debugging.
- Update the GUI to keep track of drone states and number of active fires (incidents)

Previous questions on Iteration #2:

Below are some of the questions students asked in Winter 2025 regarding iteration 2:

- For nozzle door opening times, if the nozzle is open even a little bit, the water would begin to come out. Should an assumption be made that the water does not begin to flow until the nozzle door is fully open?
 - Nozzle door opening time shouldn't be that long. You can add that time to the total time needed to release the agent.
- Should your code simulate the time that it takes for the drone to recharge and refill the tank?
 - No, you can assume that once the drone returns to the base, it is recharged and the tank is refilled and ready to be dispatched again.
- Once a drone releases its agent at an event, does it automatically return back to base, or if it still has agent, does it stay idle at the event its currently at?
 - Once it is done, it must communicate with the scheduler and see update its statues; amount of agent and remaining battery (travel time). If there is another task in the queue that the drone can service, it will continue there then returns to base. If no other tasks in the queue, it will return to base.
- Can a zone have more that a single fire at a time? If so I am guessing they can be of different severities?
 - Each zone will have one fire at a time. The CSV file provided will make sure to space the events from the same zone in a way that a drone will have time to service it.

Work Products for Iteration #2:

- “README.txt” file explaining the names of your files, set up instructions, etc.
- Breakdown of responsibilities of each team member for this iteration
- UML class diagram and sequence diagrams.

- State machine diagram for the scheduler and drone subsystems.
- Detailed set up and test instructions, including test files used
- Code (.java files, all required IntelliJ files, etc.)

Iteration 3 – Multiple Drones & Distributed Execution

For this iteration, you are to use what you have learned from Assignments 2 and 3 and split your system up into three (or more) separate programs that can run on three separate computers and communicate with each other using UDP. Your task in Assignment 3 was to implement a simple version of a remote procedure call, so you are encouraged to recycle that code here. The Scheduler will now be used to coordinate the movement of drone such that each drone services roughly the same number of zones as all of the others and so that the waiting time for fires to be extinguished in a zone is minimized. Hint: if zone 1 needs to be serviced by a drone and there is a drone on the way to service zone 2 which has the same severity and that drone must pass through zone 1, then zone 1 must be serviced instead. The state machines for each drone should execute independently of each other, but they will all have to share their location with the scheduler. The scheduler will choose which drone will be used to service a given request.

Iteration #3 Summary:

- Increase complexity: run separate programs on potentially separate machines.
- The Scheduler coordinates multiple drones, ensuring balanced workloads.
- Each drone operates independently but reports status to the Scheduler.
- Update the GUI to keep track of which drone is sent to which fire zone and the severity of each zone.

Previous questions on Iteration #3:

Below are some of the questions students asked in Winter 2025 regarding iteration 3:

- Does a drone need to be in the centre of a zone to put out the fire, or simply needs to be in the zone (at any point)?
 - Yes, once the drone reaches the centre of the zone, they will be assumed to have reached that zone and ready to release the water/foam.
- Are we required to consider drone collisions when using multiple drones?
 - It is not required. However, depending on how you are keeping track of your drones' location, you can include that in your code.
- Are we required to modify the **Drone Subsystem, Scheduler, and Fire Incident Subsystem** so that they can no longer directly call each other's methods and instead must communicate exclusively via **UDP**?
 - All communications between all three subsystems must be done using UDP.
- Do we need to demonstrate that these programs actually run on different machines, or is it sufficient to show that they communicate over a network protocol (UDP) while running on the same machine?
 - You don't need to run it on three different computers, however, you must have three different process, one process (main method) for each subsystem.
- For the unit tests, should we include testing of the UDP sending and receiving packets? If so, should we set up separate sockets within the tests to verify the packet contents?
 - For UDP communications, you need to test mainly the contents of packets sent and received. You should have methods for creating these packets. Therefore, you can test the methods directly. In the case

you don't have a specific method for creating the packets and you included it in the UDP communication method, you need to create a separate socket in within the test.

- Iterations 3 and 4 list "Any unchanged diagrams from the previous iterations UML class diagram" as a required work product. Does this mean that we do not need to provide newly updated UML Class/State/Sequence diagrams? Should only the UML class diagram be updated?
 - No new UML diagrams are needed for iteration 3 or 4. If you did any changes in the structure of your project/code, then you can update the diagrams and submit them with the solution.

Work Products for Iteration #3:

- "README.txt" file explaining the names of your files, set up instructions, etc.
- Breakdown of responsibilities of each team member for this and previous iterations
- Any unchanged diagrams from the previous iterations UML class diagram
- Detailed set up and test instructions, including test files used
- Code (.java files, all required IntelliJ files, etc.)

Iteration 4 – Fault Handling

For this iteration, you will be adding code for detecting and handling faults. To this end, you will have to add timing events so that if the timer goes off before a drone reaches a zone, then your system should assume a fault (either, the drone is stuck between zones, nozzle/bay doors are stuck or the arrival sensor at a zone has failed). Similarly, you should detect whether a nozzle/bay doors or not, or is stuck open. In Iteration 5 below, your drone status output should show these faults. However, the a nozzle/bay doors fault should be regarded as a hard fault and should shut down the corresponding drone.

You must submit code to enable us to see that your drone scheduler can deal properly with the faults shown above (i.e., you must be able to inject these faults into the system). I suggest that you inject these faults using the input file (so you will have to modify its format and be able to show to us how it works).

Iteration #4 Summary:

- Inject faults:
 - Drone stuck mid-flight
 - Nozzle jammed
 - Packet loss or corrupted messages
- The Scheduler should detect and handle these faults gracefully (e.g., reroute missions, mark a drone as offline).
- Add time stamps for events (fire detected, drone idle, drone dispatched, ...)
- Update the GUI to display faulty drone and distinguish different types of faults. Maybe use colors for different types of faults.

Previous questions on Iteration #4:

Below are some of the questions students asked in Winter 2025 regarding iteration 4:

- How should we be inputting Faults in the input file? Should they be appended to a fire event (FAULT TYPE, TIME), so if a drone gets that event it will be notified the fault in X seconds?
 - Add a column at the end of table in the input file that will be used for specifying if there should be a fault or not. That way, when a drone picks up that event, it will know that there will be a fault. Once,

depending on the fault type, the drone can either go out of commission for a short period to simulate resetting or it will go down for the remaining of the simulation run. Once the drone picks up the faulty event, that drone will have to process the fault and the event will be sent to another drone.

- Our group implemented action listener buttons in our GUI to inject faults into the system. We feel that the action listener allows for easy, interactive testing. Does this approach align with the project requirements, or should we refactor it to follow the recommendation of using an input file instead?
 - That is a good idea. However, are you able to choose the type of fault and which drone to assign the fault to? If yes, then that should be good. If not refactor your code to be able to choose and target a drone with a specific fault.

Work Products for Iteration #4:

- “README.txt” file explaining the names of your files, set up instructions, etc.
- Breakdown of responsibilities of each team member for this and previous iterations
- Any unchanged diagrams from the previous iterations UML class diagram
- Timing diagrams showing the error scenarios for this iteration
- Detailed set up and test instructions, including test files used
- Code (.java files, all required IntelliJ files, etc.)

Iteration 5 – Capacity Limits & GUI

The last stage of the project is to add a display console showing where each of the drone is in real time and displaying any faults (if any). The idea is to have output suitable for the fire fighter dispatcher sitting at the desk in headquarters to refer to. This part of the project can be part of the scheduler subsystem if you so choose, or you may want to develop a separate subsystem altogether.

You are also to implement capacity limits (if not done so previously) so that zones waiting to be serviced will have to wait for a drone with a full tank.

You are also to add instrumentation to your code to record the time it takes to extinguish all fires in the input file and to record the distance required to reach desired zone. In the real world, the goal is to minimize these numbers.

Iteration #5 Summary:

- Enforce agent capacity: if a drone runs out of firefighting agent, it must return to a “refill station.”
- Add a user interface showing each drone’s location, fire statuses, and any faults.
- Collect and log overall performance metrics (response times, fire extinguish times, etc.).
 - Calculate average drone idle time, drones flight time, time between an incident detected to being distinguished, amount of time needed to distinguish all fires (from event 1 to servicing last event), etc.
- Finalize the GUI to display different zones and incident severity, normal and faulty drones and the status of each drone (battery/fuel and amount of firefighting agents).

Previous questions on Iteration #5 and Final Project Presentation:

Below are some of the questions students asked in Winter 2025 regarding iteration 5 and Final presentation:

- Are multiple drones expected to be dispatched to the same fire incident zone if needed? For example, if Zone 1 has a high severity fire needing 30 liters, should two drones be dispatched from the origin at the same time with 15 liters each?

- This is part of your scheduling algorithm. You need to decide how to do this. You can either dispatch enough drones to one zone to extinguish the fire, or you can dispatch one drone then if there is more fire, then you send another one. If you dispatch two drones at the same time, you would need to add some delay between the two drones to avoid collisions.
- Regarding the capacity limits: if a drone finishes dealing with one fire and still has some extinguishing agent left, is it expected to assist another zone immediately (assuming it has enough solution and battery), or should it return to base and let a fully loaded drone handle the next fire?
 - If the drone has enough agent and battery (to go to the zone then go back to base) to assist with another zone, then it should continue to the next zone. You have to keep in mind that no other drones are on the way to that zone.
- Should the final deliverable have all the diagrams from each iterations (ex. iteration 1 class diagram, iteration 2 class diagram, iteration 3 class diagram...) or just the final versions of each diagram.
 - Just submit the most recent version of each of the diagrams that were required for the project.
- Under the Final Project Submission it talks about a brief video demonstrating our iteration 5 along with the final report. Are we to make our own input file as we see fit, or is there a certain scenario to be demonstrated?
 - You are required to make a 3-5 minutes video demo of your project which should be similar to what you demo to the TA during final lab. You can use the input files you submitted with iteration 5.
- Is the video a demonstration of the simulation running, showing the steps etc., or more of in detail explaining each class, showcasing progress from each iteration?
 - Just show the simulation running and explain in general any important details about the project. Ex. You have worked on a specific scheduling algorithm/method, so explain that part.
- Are all members of group required to be in the video and speaking?
 - No. It can be explained by one or more team members.

Work Products for Iteration #5:

- README.txt with instructions and file naming conventions
- Team Responsibilities breakdown
- UML Diagrams (class, state machine, sequence diagrams)
- Detailed Setup & Test Instructions (including input files)
- Source Code (.java files, IntelliJ project files)

Final Project Presentation

Your team will give a 15 minute presentation to a TA of iteration #5. It must show 20 drones running with 5 fire incident zones. Your demonstration should also show a drone stuck mid-flight and nozzle jammed on a drone. The goal of the presentation is to show us how awesome your team and project are, and to submit your final copy of your work using Brightspace.

Finally, we will have a little contest to see whose Firefighting Drone Swarm system can extinguish all fires from an input file supplied to you in the least amount of time. We shall assume that all the drones are working for the entire period (no faults).

Final Project Presentation Summary:

- A 15-minute demo of your final system.

- Must show multiple drones (10) responding to multiple zones (5 zones).
- Demonstrate at least one fault scenario (e.g., a drone stuck mid-flight) and how your Scheduler manages it.
- The goal is to showcase your system's abilities and to submit all code/documentation on Brightspace.

The winning team earns bragging rights!

Final Deliverables

You must submit a **report** containing:

1. A report consisting of:
 - Team number and team members
 - Table of contents
 - Breakdown of responsibilities of each team member for each iteration
 - All diagrams:
 - UML class diagrams for the three components.
 - A State Machine diagram for the scheduler.
 - Sequence diagrams showing all the error scenarios.
 - Timing diagrams for the scheduler.
 - Detailed set up and test instructions.
 - Results from your measurements.
 - Reflection on your design - what parts do you like and what parts should be redone.
2. Test files for all iterations.
3. Code (.java files, all required IntelliJ files, etc.)

Submission and Grading

Please refer to Brightspace for all submission dates.

Work products for each iteration are to be submitted using Brightspace. Submit using your project team's number, (one submission per team) by the date/time specified. You can include any number of files in your submission. Ensure that all your files are accepted. Please submit PDF instead of Word, LibreOffice, Pages, etc. documents.

The project is worth 30% of your final grade. Iterations #1 through #4 are worth 2.5 marks each. The final demo is worth 5 marks. 15 project marks are for the final deliverables. These 30 marks are for the entire team. However, based on the participation of each member, each individual's mark could be higher, the same, or lower than the team mark. The highest marks will be awarded to teams where each member participates and contributes roughly equally. Note that a team member who dominates the group is just as likely to lose marks as an individual who goofs off. Mentoring and team-programming is strongly encouraged. Please make sure that all criticism is constructive. I too have written some pretty crappy code in my day and have appreciated constructive feedback.

Submission and Grading summary

- Submit via Brightspace by the deadlines given.
- One submission per team; ensure all files are accepted.
- Please submit PDFs for documentation.
- The project is **30%** of your final grade, split as follows:

- Iterations #1–#4: 2.5 marks each (10 total)
- Final demo: 5 marks
- Final deliverables: 15 marks
- Individual marks can vary based on participation. Balanced contributions typically earn the highest marks.

Bi-Weekly Meetings

Each team will have an approximately biweekly 15 minute meeting with a TA to discuss their progress. The meetings are mandatory, and will be held during the labs. The TA's will expect a demo of each iteration during the meeting. This will be complicated with remote learning, but we will muddle along as best we can.

Good luck and have fun implementing your Firefighting Drone Swarm!

Graduate Attribute		Performance Level	Level 1	Level 2	Level 3	Level 4
		Level Descriptor	Beginning	Developing	Accomplished	Exemplary
6	Individual and team work	6.1 Personal and group time management	<i>Deadlines often missed. Several projects or assignments appear hurried or inadequately addressed. No evidence of time management planning.</i>	<i>Some deadlines missed. Some tasks or assignments appear hurried or inadequately addressed. Some evidence of time management planning.</i>	<i>No important deadlines missed. Few tasks or assignments appear hurried or inadequately addressed. Good time management planning.</i>	<i>No deadlines missed. Tasks and assignments always excellently prepared and presented. Tasks occasionally completed ahead of schedule. Leadership in time planning.</i>
		6.2 Group culture, group dynamics	<i>Not dependable. Little or no trust from teammates. Dishonest or evasive interactions with teammates. Poor conflict resolution.</i>	<i>Usually dependable and trusted by teammates. Mostly ethical behavior towards project and teammates. Minor conflicts.</i>	<i>Dependable and trusted by teammates. Ethical open interactions with teammates. Good conflict resolution.</i>	<i>Dependable and trusted by team mates. Ethical and open interactions with teammates. Leads team in ethical and responsive behavior. Leadership in conflict resolution.</i>
		6.3 Leadership: initiative and mentoring, areas of expertise, and interdisciplinary teams	<i>No initiative. No effort to mentor group members or take ownership of an area of expertise.</i>	<i>Some initiative. Some effort to mentor group members and take ownership of an area of expertise.</i>	<i>Good initiative and self-motivation. Regular effort to mentor group members. Clear command of an area of expertise.</i>	<i>Self-motivated, with regular demonstration of initiative. Constructively directs other team members and helps them to improve at their tasks. High degree of knowledge in an area of expertise.</i>

Change Log

1.0: Initial version.

1.1: Revise input table format

1.2: Revise zone coordinates as input from file

1.3: Added, amounts of water/foam needed for each event severity level. Also, added diagrams showing drone components and sample grid layout.

1.4: Revise iteration 04's instructions

2.1: Added details on GUI for each iteration and added QA for each iteration